
CampusRent: A Contact-Lite Campus Rental Platform

Chenze Ma, Tianqin Liu, Sipeng Wu, Hanpei Hu, Yuzhe Han, Chenyu Zhang, Xiangyu Yan
CSC3170 Database Systems Group 14
The Chinese University of Hong Kong, Shenzhen

Abstract

Many university students need temporary access to items such as cameras, tools, electronics, books, and home appliances, but purchasing these items is often unnecessary and inefficient. CampusRent addresses this problem by providing a campus-oriented rental platform that supports online browsing and reservation, centralized offline pickup and return, and database-driven operational management. The system combines a normalized relational database, MySQL-based data loading scripts, role-based security controls, masked privacy views, analytical SQL queries, and a local web user interface. The project demonstrates how a database-centered design can improve the accessibility, efficiency, and privacy of short-term resource sharing in campus scenarios.

1 Introduction and Motivation

University communities contain many items that are useful only for a short period of time. For example, students may need a camera for one project, a ladder for one move-in day, or a calculator for one course. Buying these items creates unnecessary cost and waste, while informal borrowing is hard to manage because availability, pickup, payment, and return status are not recorded in a structured way [Hamari et al., 2016].

Existing solutions such as peer-to-peer marketplaces or general rental platforms are not well suited for campus environments. They often require direct coordination between users, lack centralized inventory management, and do not provide sufficient support for tracking transactions, enforcing rules, or protecting user privacy. The informal borrowing within student communities, while convenient, further lacks accountability and transparency. Hence, we introduce CampusRent.

CampusRent is designed as a contact-lite campus rental platform. The platform allows users to browse available items online, reserve items through the system, and collect or return them at designated campus pickup locations. It also allows the user to see their profit and cost on the platform. Items input and return confirmation are handled by operators, while administrative and analytical tasks are supported through separate staff pages. This design reduces direct coordination between owners and borrowers and makes the rental process easier to track through the database. Providing a secure and solid platform for the transaction of items.

From a system perspective, managing such a rental platform requires reliable data organization, transaction consistency, and secure access control. A database-centered design enables structured storage of users, items, reservations, and transactions, while ensuring data integrity and supporting complex queries for operational and analytical purposes. This motivates the use of a relational database system as the backbone of CampusRent.

To this end, CampusRent is developed as an integrated database system rather than a standalone interface prototype. The project constructs a relational schema for users, items, categories, pickup locations, payment records, sensitive identity data, and rental orders, and implements the schema in MySQL with integrity constraints and sample data loading. On top of this database, we further provide

daily operational SQL, analytical and data-mining-style queries, role-based security controls, privacy-preserving masked views, and a local web interface for demonstrating user, operator, administrator, and analyst workflows. Through this design, CampusRent shows how a relational database can support not only basic storage, but also transaction management, privacy-aware access, and data-informed decision making in a campus rental scenario.

2 Design and Implementation

2.1 System Overview

CampusRent is a local database-supported web application designed for a campus rental scenario. Its main goal is to let students post idle items for rent, browse available items, and create rental orders through a simple web interface. Behind the interface, the system uses MySQL to manage key data such as users, sensitive identity records, payment methods, categories, locations, items, and orders [Oracle Corporation, 2026]. The sample data are loaded from CSV files through SQL scripts, which makes the system easier to test and demonstrate. The application layer provides a web interface for regular users and staff users. Regular users can log in, browse inventory, and place rental orders. Operators can add items on behalf of owners and confirm returns. Administrators can inspect the system architecture and table data. Data analysts are allowed to view authorized datasets and analysis results. Data analysts are allowed to view authorized datasets and analysis results. Overall, CampusRent is built as a small but complete database system that supports real campus rental workflows.

2.2 Core Business Logic and ER Model

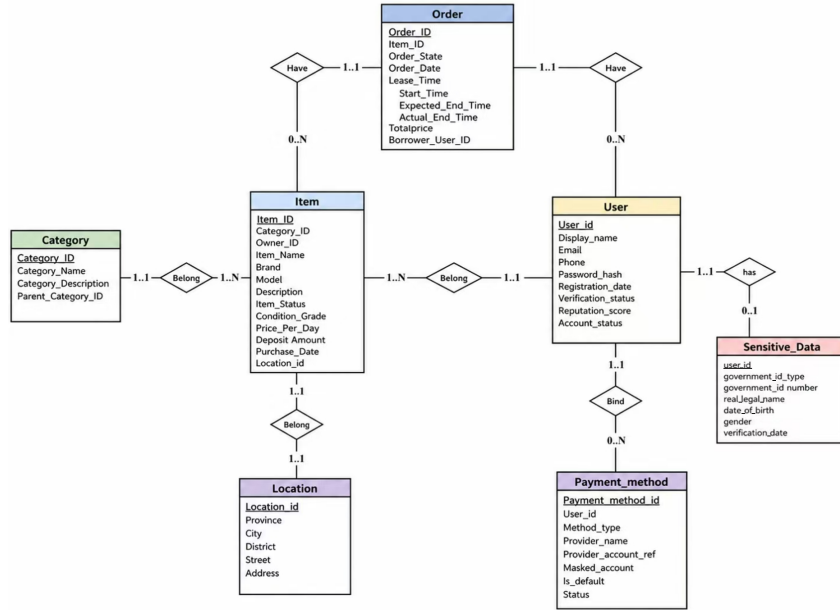


Figure 1: Conceptual ER model of the CampusRent database.

The core business logic of CampusRent can be summarized as follows: users post items, and other users place orders to rent or borrow the items. This link directly determines the structure of the ER model. The whole ER model is mainly organized around three central entities: User, Item, and Order. The database design is built around these entities and their connections with supporting entities such as Category, Location, Sensitive_Data, and Payment_Method.

Firstly, User represents the account holder on the platform. A user can either be the owner who posts an item or the borrower who rents an item. The attributes include user ID, display name, email, phone number, password hash, registration date, verification status, reputation score, and

account status. The primary key is `user_id`, which uniquely identifies each user. User is also connected to `Sensitive_Data` and `Payment_Method`. `Sensitive_Data` stores real-name verification information, while `Payment_Method` stores the user's payment-related information.

Secondly, `Item` is the core object being rented. Its attributes include item ID, item name, brand, model, description, item status, condition grade, daily rental price, deposit amount, purchase date, and location ID. The primary key is `item_id`. Each item belongs to one category and one location. At the same time, each item is posted by one user as its owner. This design allows the platform to clearly record what the item is, who owns it, where it is located, and what category it belongs to.

Thirdly, `Order` saves a complete historical rental transaction. Its attributes include order ID, item ID, order state, order date, lease time, start time, expected end time, actual end time, total price, and borrower user ID. The primary key is `order_id`. Each order is linked to one item and one borrower. This means an item can appear in many orders over time, but each specific order only refers to one item. Similarly, one user can place many orders, but each order has one specific borrower. This relationship supports the basic rental process of the platform, from item browsing to order creation and return confirmation.

In addition to these three core entities, there are four supporting entities that make the system more complete. `Category` is used to classify items and supports hierarchical categories. `Location` stores address information separately, which avoids repeated address data in the `Item` table. For privacy protection, `Sensitive_Data` separates real identity information from the main `User` table, so external attackers cannot directly obtain users' real sensitive identity information. `Payment_Method` saves users' payment methods and allows one user to bind multiple payment options.

2.3 Relational Schema Design

Item (<u>item_id</u>, category_id ^{FK}, owner_id ^{FK}, item_name, brand, model, description, item_status, condition_grade, price_per_day, deposit_amount, purchase_date, location_id ^{FK}) PK: item_id Foreign keys: - category_id references Category.category_id - owner_id references User.user_id - location_id references Location.location_id	User (<u>user_id</u>, display_name, email, phone, password_hash, registration_date, verification_status, reputation_score, account_status, deposit) PK: user_id UNIQUE: email, phone
Category (<u>category_id</u>, category_name, category_description, parent_category_id ^{FK}) PK: category_id FK: parent_category_id references Category.category_id	Sensitive_Data (<u>user_id</u> ^{FK}, government_id_type, government_id_number, real_legal_name, date_of_birth, gender, verification_date) PK: user_id FK: user_id references User.user_id UNIQUE: government_id_number
Location (<u>location_id</u>, province, city, district, street, address) PK: location_id	Payment_Method (<u>payment_method_id</u>, user_id ^{FK}, method_type, provider_name, provider_account_ref, masked_account, is_default, status, created_at) PK: payment_method_id FK: user_id references User.user_id
Order (<u>order_id</u>, item_id ^{FK}, borrower_user_id ^{FK}, order_state, order_date, lease_time, start_time, expected_end_time, actual_end_time, total_price) PK: order_id Foreign keys: - item_id references Item.item_id - borrower_user_id references User.user_id	

Figure 2: Relational schema of the CampusRent database.

After finishing the ER model, we converted it into a relational schema so that the design could be implemented in MySQL. The main entities in the ER model were turned into tables, and the relationships between them were represented by primary keys and foreign keys. The schema contains seven main tables: `Category`, `Location`, `User`, `Sensitive_Data`, `Payment_Method`, `Item`, and `Order`.

The `User` table is used to store the basic account information of platform users. Its primary key is `user_id`, which allows the system to identify each user record clearly. Email and phone number were set as unique fields for the importance of login, contact, and account verification. This prevents two users from registering with the same email address or phone number. For sensitive identity information, we created a separate `Sensitive_Data` table. This table also uses `user_id` as its primary key, and at the same time, `user_id` is a foreign key referencing `User(user_id)`. In this way, each user can have at most one corresponding sensitive-data record, which forms a one-to-one relationship. The `government_id_number` field is also set as unique to avoid duplicated identity verification records.

The `Payment_Method` table is designed to store users' payment information separately. Its primary key is `payment_method_id`, and it uses `user_id` as a foreign key to connect back to the `User` table. This means that one user may bind more than one payment method. Payment methods may change or increase over time, while the user's basic account information should remain relatively stable.

The `Item` table is the central table for the rental function. Its primary key is `item_id`. The table is connected to three other tables through foreign keys. The field `category_id` references `Category(category_id)`, which tells the system what type of item it is. The field `owner_id` references `User(user_id)`, which records who posted or owns the item. The field `location_id` references `Location(location_id)`, which shows where the item is located. Through these links, the `Item` table does not only store item details, but also connects the item with its owner, category, and physical location.

The `Order` table records the actual rental activities on the platform. Its primary key is `order_id`. It contains two important foreign keys: `item_id` and `borrower_user_id`. The field `item_id` references `Item(item_id)`, so each order can be matched with the item being rented. The field `borrower_user_id` references `User(user_id)`, so the system can identify the user who places the order. This design matches the core business process of CampusRent: a user chooses an item and creates an order to rent it. The table also stores information such as order state, rental time, return time, and total price, which are necessary for tracking the whole rental process.

Overall, the relational schema follows the ER model while preparing it for practical database implementation. Primary keys identify records, and foreign keys maintain consistency between related tables. By separating users, sensitive data, payment methods, items, locations, categories, and orders, the schema reduces redundancy and supports key CampusRent workflows such as registration, item posting, browsing, order creation, return confirmation, and later analytical queries.

Table 1: Main relations in the CampusRent database.

Relation	Primary Key	Foreign Keys	Purpose
user	user_id	–	Stores routine account and reputation data.
sensitive_data	user_id	user_id → user	Stores legal identity information separately.
payment_method	payment_method_id	user_id → user	Stores payment provider references.
category	category_id	parent_category_id → category	Stores item category hierarchy.
location	location_id	–	Stores pickup and return locations.
item	item_id	owner_id → user category_id → category location_id → location	Stores rentable inventory.
order	order_id	item_id → item borrower_user_id → user	Stores rental transactions and rental status.

2.4 Data Loading and Implementation

The database is implemented in MySQL through the initialization script `database/init.sql`. The script first creates the database, drops old tables if they already exist, defines all relational tables, and sets up primary keys, foreign keys, and unique constraints.

Instead of writing hundreds of manual `INSERT` statements, we store the sample data in CSV files and use `LOAD DATA LOCAL INFILE` for bulk import. This approach makes the data easier to inspect, modify, regenerate, and reload. The dataset is organized by entity as follows:

Entity / Table	CSV File	Number of Records
category	category.csv	59
location	location.csv	4
user	USER.csv	300
sensitive_data	sensitivedata.csv	300
payment_method	payment.csv	446
item	item.csv	305
order	order.csv	540
Total	–	1,954

This dataset covers all major entities required by the CampusRent platform, including users, sensitive identity records, payment methods, item categories, pickup locations, rentable items, and rental orders. Since each CSV file corresponds to one major table in the database, the data-loading process is consistent with the relational schema. To further illustrate how the imported CSV records are connected across the database, we use user U000039 as an example. This example shows one user's account information, sensitive identity record, payment method, owned item, item category, item location, and borrowing history after the CSV files are loaded into MySQL.

Relation	Real Data Example
user	U000039, jiho_58, jiho.sato000039@example.com, +1-555-842-7916, sha256\$7208380ee2f44122baffcc20, 2023-03-19 00:00:00, verified, 4.66, active, \$300.00
sensitive_data	U000039, national_id, N10001443, Jiho Sato, 1997-01-25 00:00:00, female, 2023-04-28 00:00:00
payment_method	PM052, U000039, paypal, PayPal, acct_k9jos4u6, ji***@example.com, True, active, 2025-02-18 20:30:00
category	C000031, Cookware, Pots and pans for everyday cooking, C000030
item	I000293, C000031, U000039, GreenPan Saucepan, GreenPan, Padova 3Qt, ceramic-coated saucepan, available, A, \$5.00, \$15.00, 2024-01-18 00:00:00, L000002
location	L000002, California, Palo Alto, Stanford University, Main Quad, 450 Serra Mall
order	0000204, I000251, U000039, completed, 2025-02-25 00:00:00, 5, 2025-04-06 12:30:00, 2025-04-11 12:30:00, 2025-04-11 20:30:00, \$20.00 0000369, I000060, U000039, completed, 2025-09-22 00:00:00, 3, 2025-10-18 16:00:00, 2025-10-21 16:00:00, 2025-10-21 20:00:00, \$18.00

The CSV files are loaded in an order that follows the dependency structure of the schema. Parent tables such as category, location, and user are loaded first, because later tables depend on them through foreign keys. For example, item records must be loaded after category, user, and location, since each item refers to a category, an owner, and a location.

All CSV files are imported using the same general pattern: LOAD DATA LOCAL INFILE. The following example shows how item.csv is loaded into the item table:

```
LOAD DATA LOCAL INFILE 'item.csv'
INTO TABLE item
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(item_id, category_id, owner_id, item_name, brand, model, description,
 item_status, condition_grade, @price, @dep, @purchase, location_id)
SET price_per_day = CAST(REPLACE(@price, '$', '') AS DECIMAL(10,2)),
    deposit_amount = CAST(REPLACE(@dep, '$', '') AS DECIMAL(10,2)),
    purchase_date = STR_TO_DATE(NULLIF(@purchase, ''), '%Y/%c/%e');
```

This example also illustrates the data-cleaning operations used during loading. Currency strings such as \$5.00 are converted into numeric DECIMAL values by removing the dollar sign and applying CAST. Date strings are converted into SQL DATE values using STR_TO_DATE. In other tables, the same strategy is used to convert datetime strings, boolean values, and empty strings into proper SQL formats.

After the loading process, the script performs a sanity check by counting the number of rows in each table:

```
SELECT 'category' AS table_name, COUNT(*) AS row_count FROM category
UNION ALL SELECT 'location', COUNT(*) FROM location
UNION ALL SELECT 'user', COUNT(*) FROM user
UNION ALL SELECT 'sensitive_data', COUNT(*) FROM sensitive_data
UNION ALL SELECT 'payment_method', COUNT(*) FROM payment_method
UNION ALL SELECT 'item', COUNT(*) FROM item
UNION ALL SELECT 'order', COUNT(*) FROM 'order';
```

This sanity check confirms whether all CSV files were imported into the expected tables. Together with the foreign-key constraints, it helps ensure that the loaded data is complete and logically consistent.

2.5 Index and Hash Recommendations

We carefully selected which fields to hash and which to index based on the actual query patterns of the CampusRent platform.

2.5.1 Design Rationale

Hash indexes are applied to columns that are queried only with *exact equality*. These include primary keys, foreign keys used in joins, login credentials (email, phone), status columns (e.g., `item_status`, `order_state`), and unique identifiers such as `government_id_number`. A hash index delivers near $O(1)$ lookup performance for equality comparisons, making it the optimal choice for such fields.

B-Tree indexes are used for columns that support *range queries, sorting, or partial matching*. Typical examples include timestamp columns (`order_date`, `registration_date`) for reporting over periods, numeric price columns (`price_per_day`) for range filters, and time columns (`start_time`, `expected_end_time`) for checking availability conflicts. B-Tree enables efficient ordering, range scans, and prefix searches – operations that hash indexes cannot perform efficiently [Oracle Corporation, 2026].

2.5.2 Field Classification by Table

Table 2 summarises the indexing strategy for each relation.

Table 2: Hash vs. B-Tree Indexes per Table

Table	Hash Fields (Equality)	B-Tree Fields (Range / Sorting)
users	user_id (PK), email, phone	registration_date, reputation_score
items	item_id (PK), owner_id, location_id, status	price_per_day, purchase_date
orders	order_id (PK), borrower_user_id, item_id, order_state	order_date, start_time, expected_end_time, total_price
payment_methods	payment_method_id (PK), user_id, status	—
sensitive_data	user_id (PK/FK), government_id_number	verification_date
categories	—	category_name

Note: PK = Primary Key, FK = Foreign Key. For static tables like categories, only sorting support is needed.

2.5.3 Justification Examples

- `users.email` and `users.phone`: Login and password reset use exact matches. Hash is ideal.
- `items.item_status`: Frequent filtering by 'available' – equality only.
- `orders.order_date`: Monthly revenue reports require range queries (e.g., between first and last day of month). B-Tree is necessary.
- `orders.start_time` and `expected_end_time`: Conflict detection uses conditions like `start_time <= now() AND expected_end_time >= now()` – range comparisons.
- `government_id_number`: Uniqueness check and exact lookup; hash speeds up insertion validation.

2.6 Daily Operation Workflows

The operational design of CampusRent is organized around the lifecycle of a rental item: account registration, inventory intake, item discovery, order creation, offline pickup and return, and post-transaction status maintenance. Each workflow corresponds to a controlled sequence of relational operations, so the platform behavior can be represented by explicit changes in database state rather than by informal manual records.

2.6.1 User Registration and Account Records

A regular user first enters the platform through account registration or email login. At the database level, routine account attributes are stored in `users`, while identity-related and payment-related records are stored separately in `sensitive_data` and `payment_methods`. This separation allows the platform to maintain a complete user profile without requiring every operational query to access sensitive attributes. During normal rental activities, the user identifier serves as the stable reference that links accounts to owned items, payment methods, and borrowing orders.

2.6.2 Inventory Intake and Listing Management

CampusRent adopts an operator-managed inventory model. When a user wants to list an item for rental, the item is brought to a designated campus pickup location. An operator then creates the corresponding inventory record in `items`, linking it to the owner, category, and location through foreign keys. The item is assigned an initial availability status and becomes part of the searchable inventory. This design ensures that listings are created through a controlled process and that each item is associated with a real pickup location before it is shown to borrowers.

2.6.3 Browsing and Order Creation

Borrowers browse available inventory through queries over `items`, `categories`, and `locations`. These queries allow the platform to present item descriptions, prices, categories, and pickup locations. When a borrower places an order, a new record is inserted into `orders`. The order stores the selected item, borrower, rental period, expected return time, lease duration, total price, and order state. Thus, the rental agreement is represented as a transaction record that connects the borrower, the item, and the time interval of use.

2.6.4 Pickup, Return, and Status Transitions

After an order is created online, the borrower collects the item at the selected campus location. The next major database transition occurs when the item is returned. The operator confirms the return, updates the corresponding `orders` record with the actual end time, and sets the final order state to either completed or late-return depending on whether the return occurs after the expected end time. The related item can then be returned to available status or marked for maintenance if further inspection is needed. In this way, physical item circulation is synchronized with database state transitions.

2.6.5 Administrative and Analytical Operations

In addition to transactional workflows, CampusRent supports administrative and analytical database operations. Administrators can inspect table contents, schema-level relationships, and platform-level records for management purposes. Data analysts run read-only analytical queries over approved tables and masked views to evaluate category performance, location demand, item utilization, high-value borrowers, high-risk borrowers, and cancellation or late-return patterns. These workflows show that the same relational database supports both daily operations and higher-level decision making.

2.7 Analytical and Data Mining Queries

Beyond supporting transactional operations, CampusRent is designed to show how a relational database can support operational analytics and lightweight data-mining tasks. We therefore implement six analytical SQL queries in `analytic_queries.sql`. These queries are written directly on top of the final MySQL schema, so the analysis is executable on the same database used by the rental platform.

The analytical workload covers three levels of insight. First, aggregate queries summarize platform-level demand and revenue. Second, ranking queries identify important entities such as high-value borrowers and under-utilized items. Third, segmentation queries approximate data-mining behavior by grouping users and orders into interpretable behavioral bands. These queries use joins, grouping, conditional aggregation, derived metrics, ranking, and rule-based labels to transform raw rental records into decision-support information.

Due to space limitations, the full SQL statements are provided in Appendix D. The following paragraphs explain the objective, logic, and expected use of each query.

Query 1: Category Revenue and Order Volume Analysis

The first query analyzes platform performance at the item-category level. It joins `orders`, `items`, and `categories`, then groups orders by category. For each category, the query reports the number of valid orders, total order value, and average order value. The result is ordered by total order value and order count, allowing the platform administrator to identify the most profitable and most frequently rented item categories. This can support decisions such as which categories should receive more listings, promotion, or inventory balancing.

Query 2: Location Demand Distribution Analysis

The second query measures rental demand across campus pickup locations. It joins `orders`, `items`, and `locations`, because each order is linked to an item, and each item is associated with a physical location. The query groups results by location and reports order requests, distinct borrowers, requested lease days, and requested order value.

Unlike Query 1, this query does not exclude cancelled orders. This is intentional: for location-demand analysis, even cancelled orders may indicate attempted demand or user interest in a location. The result can help the platform understand which campus areas have higher rental activity and whether inventory should be redistributed across locations.

Query 3: Item Utilization and Idle Inventory Detection

The third query estimates item-level utilization and identifies potentially idle inventory. It uses a `LEFT JOIN` from `items` to `orders` so that items with no related orders are still included in the result. For each item, the query calculates total orders, booked rental days, realized revenue, and the most recent non-cancelled rental activity.

The query uses conditional aggregation to separate useful rental activity from cancelled orders. Realized revenue is counted only for orders whose state is `completed` or `late_return`. The most recent rental activity is computed from the available time attributes using `COALESCE`, including `actual_end_time`, `expected_end_time`, `start_time`, and `order_date`.

Finally, the query assigns each item a rule-based utilization label. If more than 30 days have passed since the last non-cancelled rental activity, the item is marked as `idle`. If the item has at most one non-cancelled order, it is marked as `low_utilization`. Otherwise, it is marked as `active`. This query therefore performs a simple data-mining-style classification based on historical rental behavior.

Query 4: High-Value Borrower Identification

The fourth query ranks borrowers by their historical rental value. It joins `vw_users_masked` with `orders`, where the masked user view is used instead of the original user table to avoid exposing sensitive user information in analytical results.

For each borrower, the query reports reputation score, total orders, booked value, average lease time, late-return count, and cancellation count. Booked value and average lease time are calculated only from non-cancelled orders. The result is ordered by booked value and total order count, making it possible to identify borrowers who contribute more economic value to the platform. These users may be candidates for loyalty benefits, priority service, or targeted recommendations.

Query 5: High-Risk Borrower Identification

The fifth query identifies potentially risky borrowers using relative behavioral rates rather than raw counts. This is important because users with more total orders naturally have more chances to produce problematic orders. The query therefore computes cancellation rate and late-return rate by dividing the corresponding event counts by the borrower's total order count. It also computes a rule-based `risk_score`. Cancellation rate, late-return rate, and a reputation penalty are combined into a single score. Borrowers with reputation scores below 4.2 receive an additional penalty. This design makes the query more robust than simply sorting users by the number of cancelled or late-return orders.

Query 6: Late-Return and Cancellation Pattern Mining

The sixth query performs a simple pattern-mining analysis by segmenting orders according to borrower reputation and lease duration. It first uses a common table expression, `labeled_orders`, to assign each order two labels: `reputation_band` and `lease_band`. Reputation is divided into `high_rep`, `mid_rep`, and `low_rep`, while lease duration is divided into `short_lease`, `medium_lease`, and `long_lease`.

The outer query then groups orders by these two labels and calculates total orders, cancellations, late returns, cancellation rate, and late-return rate for each segment. Although this is not a machine-learning model, it demonstrates a core data-mining idea: discovering interpretable behavioral patterns from historical transaction data. For example, the result can reveal whether low reputation users with long lease durations are more likely to cancel or return items late.

Overall, these six analytical queries demonstrate that CampusRent is not only a transactional rental platform, but also a database system capable of supporting evidence-based operational decision making.

2.8 Security and Privacy Design

CampusRent handles several categories of privacy-sensitive information, including user account attributes, legal identity records, payment-provider references, and rental behavior. Although the system is implemented as a course prototype, these data categories reflect realistic privacy risks in a rental platform: an operator may need to verify an owner or process an item return, while a data analyst may need to study demand patterns without observing raw identity or payment details. Therefore, our security design is not treated as an interface-level add-on, but as a database-level requirement. The overall objective is to preserve the usefulness of the database for daily operation and analysis while minimizing unnecessary exposure of sensitive attributes.

2.8.1 Threat Model and Design Principles

The main threat considered in this project is excessive internal access. In other words, even legitimate platform roles should not automatically receive access to all stored data. For example, a data analyst requires order and inventory records for analysis, but does not require raw government identity numbers. Similarly, an operator may need to locate a user for item intake or return confirmation, but does not require unrestricted administrative access to the whole database.

To address this threat, the implementation follows two principles. First, the database adopts a defense-in-depth design: sensitive data is separated at the schema level, exposed through controlled views, and protected again by database permissions and application-level role checks. Second, the system follows the principle of least privilege [OWASP Foundation, 2026]: each role is granted only the permissions required for its operational responsibility. This design avoids relying solely on hidden UI buttons or informal usage rules.

2.8.2 Separation of Sensitive and Operational Data

At the schema level, CampusRent separates routine operational data from sensitive private records. The `users` table stores general account information such as display name, email, phone number, reputation score, verification status, and account status. In contrast, `sensitive_data` stores legal identity attributes, while `payment_methods` stores payment-provider references. Rental inventory and transactions are stored separately in `items` and `orders`.

This separation reduces the exposure surface of common queries. Most ordinary platform operations, such as browsing items, creating rental orders, checking inventory status, or computing analytical summaries, do not need to access raw identity records. As a result, the database can restrict access to `sensitive_data` without blocking the main functionality of the rental platform. From a relational design perspective, this also improves modularity: account management, identity verification, payment handling, inventory management, and rental transactions are represented as distinct concerns.

Table 3: Least-privilege role model in CampusRent.

Role	Main responsibility	Database access scope
admin	Platform administration, schema inspection, and management review	Full access to the rental platform database.
app_user	Regular user actions such as browsing inventory, creating orders, and managing ordinary account records	Limited access to public platform tables and ordinary order/account operations.
operator	Offline item intake, return confirmation, and inventory maintenance	Access to item, order, category, location, and controlled user lookup data needed for operation.
data_analyst	Read-only reporting and analytical SQL execution	Read-only access to categories, locations, items, orders, and masked views.

2.8.3 Privacy-Preserving Views

To support controlled access, the database defines several privacy-preserving views. The view `vw_users_masked` exposes user-level information with masked contact fields. The view `vw_payment_methods_masked` provides payment-method summaries without revealing raw provider references. The view `vw_user_identity_masked` provides a restricted representation of identity information when identity-level inspection is needed.

These views form a privacy layer between raw tables and downstream tasks. For example, the analytical SQL queries are written against masked user and payment views where appropriate, rather than directly reading sensitive base tables. This allows the analyst role to compute high-value borrower rankings, risk rates, and rental behavior patterns while avoiding direct exposure to raw private values. Thus, the database remains analytically useful, but the amount of personal information available to non-administrative roles is deliberately reduced.

2.8.4 Role-Based Access Control

CampusRent implements role-based access control using MySQL roles and grants [Oracle Corporation, 2026]. The final role model contains four roles: `admin`, `app_user`, `operator`, and `data_analyst`. Their access rights are aligned with the business workflow of the platform, as summarized in Table 3.

This role design reflects the actual division of labor in the platform. Regular users interact with the rental marketplace online. Operators handle physical inventory intake and return confirmation at pickup locations. Data analysts inspect aggregate behavior and operational patterns. Administrators retain full system access for database management and review. By mapping each business role to a separate database role, the system makes permissions explicit and auditable.

2.8.5 Database Accounts and Account-Level Controls

In addition to abstract roles, the implementation creates concrete local MySQL accounts, including `app_local`, `operator_local`, `analyst_local`, and `admin_local`. Each account is assigned a default role corresponding to its purpose. This means that access restrictions are enforced by MySQL itself: if a role has not been granted direct access to a sensitive table, a query against that table is rejected even if it is manually submitted outside the UI.

The security setup also includes account-level controls appropriate for a demonstration database. These include strong demonstration passwords, password expiration intervals, failed-login attempt limits, and temporary lockout settings. The schema further constrains stored user passwords to follow a hashed format [JowaspCheatSheets]. Although these mechanisms are simplified compared with a production authentication system, they demonstrate the intended database security posture: authentication, authorization, and sensitive-data access are handled explicitly rather than left implicit.

2.8.6 Application-Level Enforcement

The web interface follows the same role model as the database. Regular users enter through the email login page and access user-facing pages such as browsing, ordering, and rental history. Staff users enter through a separate staff portal. After successful staff login, the application stores the actor type and staff role in the session. Restricted staff routes then check the required role before serving the page.

This application-level enforcement is consistent with the database-level grants. For example, the operator console is designed for inventory intake, return confirmation, and maintenance status updates; the analyst console exposes read-only analytical datasets and query results; and the admin console provides schema and table inspection. If a user attempts to access a page outside the assigned role, the application redirects the request back to the staff login page. Therefore, access control is not implemented in only one layer. The UI guides users to the correct workflow, while the database grants provide a stronger underlying permission boundary.

2.8.7 Security Implication

The resulting design balances usability and privacy. Daily rental workflows can still be executed by regular users and operators, and analytical SQL can still produce useful platform insights. At the same time, sensitive identity and payment information is separated, masked, and protected by role-specific database permissions. This demonstrates that even in a small database prototype, privacy protection can be incorporated into the schema design, query interface, role model, and application workflow rather than being treated as an afterthought.

2.9 User Interface

The user interface is implemented as a local database-backed web application for demonstration and validation purposes. Rather than serving as an independent front-end mock-up, the UI connects directly to the MySQL database and exposes the main relational workflows through role-specific pages. Its primary function is to make the database design observable: users can trigger ordinary rental actions, staff members can manage operational state transitions, administrators can inspect the schema, and analysts can view approved analytical results.

2.9.1 User-Facing Interface

The regular user interface provides the public entry point of CampusRent. Users log in with email and password, browse available inventory, inspect item details, place rental orders, and review their rental activity. These pages are designed to demonstrate that the relational schema supports a realistic online rental experience. For example, the browsing pages combine item, category, owner, and location information, while order pages expose the transaction records associated with the current user. The UI therefore translates relational data into a borrower-facing workflow without requiring users to interact with SQL directly.

2.9.2 Staff Portal

Internal operations are separated from ordinary user actions through a staff portal. The staff portal routes authenticated staff members to role-specific consoles. The operator console provides controls for inventory intake, return confirmation, and maintenance status updates. The admin console provides a database-oriented view of the system, including the ER diagram, relational schema information, foreign-key relationships, and paginated table previews. The analyst console presents approved read-only datasets and the results of the analytical SQL queries. This separation makes the role model visible in the interface and keeps operational, administrative, and analytical actions distinct.

2.9.3 Interface as System Demonstration

The UI also serves as an integrated demonstration of the project requirements. It shows that the implemented schema is not merely a static design, but can support end-to-end platform behavior. The admin pages make the database structure and relationships inspectable, while the analyst pages connect the analytical SQL component to visible query results. At the same time, staff-only pages enforce role-aware navigation, so users are directed only to the workflows appropriate to their role.

In this way, the interface demonstrates the interaction between database implementation, application workflow, access control, and analytical functionality in a single local prototype.

3 Conclusion and Self-Evaluation

CampusRent demonstrates a complete database-driven solution for campus item rental. The project covers requirement analysis, ER modeling, relational schema design, MySQL implementation, data loading, operational SQL, analytical SQL, security and privacy controls, and a working local UI. The design shows that a relational database can support both daily transaction management and higher level operational analysis.

The main strengths of the project are the contact-lite workflow, the separation between regular user and staff operations, the use of masked views for privacy, and the integration between SQL logic and UI behavior. The analytical queries also make the dataset more useful by turning raw orders into business insights.

There are still limitations. The platform is currently a local prototype rather than a deployed online service. Password handling and authentication are suitable for a course demonstration but would need stronger production-grade security in a real system. In addition, the sample dataset is generated for demonstration, so future work could evaluate the platform with real campus usage data and improve recommendation, fraud detection, and demand forecasting functions.

Looking ahead, while the current CampusRent platform is designed for a university campus, we envision expanding the service to cover an entire city or larger regions. With ten to one hundred times growth in data volume, the architecture would need to evolve: horizontal sharding by city or region to keep queries local, read-write separation with multiple replicas, Redis caching for hot inventory, geospatial indexes (GIS) to support “find items near me” queries, and distributed transaction management with eventual consistency for operations spanning multiple shards. These enhancements would provide linear scalability, low latency for geographic searches, high availability, and the ability to transform CampusRent into a regional sharing economy platform.

References

- Juho Hamari, Mimmi Sjöklint, and Antti Ukkonen. The sharing economy: Why people participate in collaborative consumption. *Journal of the Association for Information Science and Technology*, 67 (9):2047–2059, 2016. doi: 10.1002/asi.23552.
- Oracle Corporation. *MySQL 8.4 Reference Manual*, 2026. URL <https://dev.mysql.com/doc/refman/8.4/en/>.
- OWASP Foundation. *OWASP Cheat Sheet Series*, 2026. URL <https://cheatsheetseries.owasp.org/>.

Appendix

A File Organization and Running Instructions

The submitted project files are organized as follows:

- data/csv/: CSV files used to load the sample database;
- database/init.sql: database creation, table definitions, constraints, indexes, and CSV loading;
- database/security_setup.sql: roles, grants, password checks, and masked views;
- sql/analytics/analytic_queries.sql: six analytical and data-mining-style queries;
- sql/operations/: sample SQL files for daily platform operations;
- ui/: local web UI and one-click launcher;
- docs/: project documents, presentation slides, and report files.

The database can be initialized in MySQL by running the initialization SQL file, then applying the security setup SQL file. Detailed local setup instructions are provided in the project README.

B Selected SQL Files

The core SQL files should be submitted together with the report or included as additional appendices:

- database/init.sql
- database/security_setup.sql
- sql/analytics/analytic_queries.sql
- sql/operations/buyer_queries.sql
- sql/operations/borrower_queries.sql
- sql/operations/platform_queries.sql

C Daily Operation SQL Queries

This appendix lists all SQL queries that support daily platform operations for owners (lenders), renters (borrowers), and platform management. These queries are taken directly from the implementation files under sql/operations/ and are used in the web interface and staff tools.

Owner (Lender) Queries

```
INSERT INTO user (
  user_id, display_name, email, phone, password_hash,
  registration_date, verification_status, reputation_score,
  account_status, deposit
) VALUES (
  'U010001', 'New User', 'newuser@example.com', '+1-555-123-4567',
  'sha256$randomhash12345678', NOW(), 'pending', 3.5, 'active', 100
);
```

Listing 1: Create a new user account

```
INSERT INTO item (
  item_id, category_id, owner_id, item_name, brand, model,
  description, item_status, condition_grade, price_per_day,
  deposit_amount, purchase_date, location_id
) VALUES (
  'I010001', 'C010001', 'U010001', 'Canon EOS R6', 'Canon', 'EOS R6',
  'Full-frame mirrorless camera', 'available', 'A', 50.00, 200.00,
  '2023-01-15', 'L010001'
);
```

Listing 2: List a new item for rent

```
SELECT
    u.user_id,
    u.display_name,
    IFNULL(SUM(o.total_price), 0) AS total_income_this_month
FROM user u
LEFT JOIN item i ON u.user_id = i.owner_id
LEFT JOIN 'order' o ON i.item_id = o.item_id
    AND o.order_state IN ('completed', 'late_return')
    AND o.order_date BETWEEN DATE_FORMAT(CURDATE(), '%Y-%m-01')
        AND LAST_DAY(CURDATE())
WHERE u.user_id = 'U000001'
GROUP BY u.user_id, u.display_name;
```

Listing 3: Query total rental income for the current month

Renter (Borrower) Queries

```
SELECT
    user_id, display_name, email, phone,
    reputation_score, deposit, account_status,
    verification_status, registration_date
FROM user
WHERE user_id = 'U000010';
```

Listing 4: View personal profile

```
SELECT
    i.item_id, i.item_name, i.brand, i.price_per_day,
    l.city, l.district, l.address
FROM item i
LEFT JOIN location l ON i.location_id = l.location_id
WHERE i.item_status = 'available'
ORDER BY i.price_per_day DESC;
```

Listing 5: Display currently available items

```
SELECT
    i.item_id, i.item_name,
    l.province, l.city, l.district, l.street, l.address
FROM item i
LEFT JOIN location l ON i.location_id = l.location_id
ORDER BY i.item_id;
```

Listing 6: Query item locations

```
SELECT
    o.order_id, i.item_name, o.order_state, o.order_date,
    o.start_time, o.expected_end_time, o.total_price
FROM 'order' o
JOIN item i ON o.item_id = i.item_id
WHERE o.borrower_user_id = 'U000010'
ORDER BY o.order_date DESC;
```

Listing 7: View order history

```
SELECT
    o.order_id, i.item_name, o.start_time,
    o.expected_end_time, i.price_per_day
FROM 'order' o
JOIN item i ON o.item_id = i.item_id
```

```
WHERE o.borrower_user_id = 'U000010'
AND o.order_state = 'active'
ORDER BY o.expected_end_time;
```

Listing 8: View currently active rental orders (not yet returned)

```
SELECT
    u.user_id,
    u.display_name,
    IFNULL(SUM(o.total_price), 0) AS total_expense_this_month
FROM user u
LEFT JOIN 'order' o ON u.user_id = o.borrower_user_id
AND o.order_state IN ('completed', 'late_return')
AND o.order_date BETWEEN DATE_FORMAT(CURDATE(), '%Y-%m-01')
AND LAST_DAY(CURDATE())
WHERE u.user_id = 'U000010'
GROUP BY u.user_id, u.display_name;
```

Listing 9: Calculate total expenses for the current month

```
SELECT
    payment_method_id, method_type, provider_name,
    masked_account, is_default, status, created_at
FROM payment_method
WHERE user_id = 'U000001'
ORDER BY is_default DESC, created_at DESC;
```

Listing 10: Query all payment methods of a user

Platform Management Queries

```
SELECT
    SUM(total_price) AS total_revenue_this_month
FROM 'order'
WHERE order_state = 'completed'
AND order_date BETWEEN DATE_FORMAT(CURDATE(), '%Y-%m-01')
AND LAST_DAY(CURDATE());
```

Listing 11: Calculate total platform revenue for the current month

```
SELECT
    i.item_id, i.item_name,
    u.display_name AS owner_name, u.email, u.phone
FROM item i
JOIN user u ON i.owner_id = u.user_id
WHERE i.item_id = 'I000001';
```

Listing 12: Retrieve contact information of an item's owner

```
SELECT
    o.order_id,
    u.display_name AS borrower_name,
    i.item_name,
    o.expected_end_time,
    TIMESTAMPDIFF(HOUR, o.expected_end_time, NOW()) AS overdue_hours,
    o.order_state
FROM 'order' o
JOIN user u ON o.borrower_user_id = u.user_id
JOIN item i ON o.item_id = i.item_id
WHERE o.order_state IN ('active', 'late_return')
AND o.actual_end_time IS NULL
AND o.expected_end_time < NOW()
ORDER BY o.expected_end_time;
```

Listing 13: Find overdue orders (expected end time passed and not yet returned)

D Analytical SQL Queries

This appendix lists the six SQL queries implemented in `analytic_queries.sql`.

```
SELECT
    c.category_name,
    COUNT(o.order_id) AS order_count,
    ROUND(SUM(o.total_price), 2) AS total_order_value,
    ROUND(AVG(o.total_price), 2) AS avg_order_value
FROM orders o
JOIN items i ON i.item_id = o.item_id
JOIN categories c ON c.category_id = i.category_id
WHERE o.order_state <> 'cancelled'
GROUP BY c.category_id, c.category_name
ORDER BY total_order_value DESC, order_count DESC;
```

Listing 14: Query 1: Category revenue and order volume analysis

```
SELECT
    l.province,
    l.city,
    COUNT(o.order_id) AS order_requests,
    COUNT(DISTINCT o.borrower_user_id) AS distinct_borrowers,
    SUM(o.lease_time) AS requested_lease_days,
    ROUND(SUM(o.total_price), 2) AS requested_order_value
FROM orders o
JOIN items i ON i.item_id = o.item_id
JOIN locations l ON l.location_id = i.location_id
GROUP BY l.location_id, l.province, l.city
ORDER BY order_requests DESC, requested_order_value DESC;
```

Listing 15: Query 2: Location demand distribution analysis

```
SELECT
    i.item_id,
    i.item_name,
    c.category_name,
    i.item_status,
    COUNT(o.order_id) AS total_orders,
    COALESCE(SUM(CASE WHEN o.order_state <> 'cancelled' THEN o.lease_time END), 0) AS
    booked_days,
    ROUND(COALESCE(SUM(CASE WHEN o.order_state IN ('completed', 'late_return') THEN o.
    total_price END), 0), 2) AS realized_revenue,
    MAX(
        CASE
            WHEN o.order_state <> 'cancelled'
            THEN COALESCE(o.actual_end_time, o.expected_end_time, o.start_time, o.order_date)
        END
    ) AS last_rental_activity,
    CASE
        WHEN MAX(
            CASE
                WHEN o.order_state <> 'cancelled'
                THEN COALESCE(o.actual_end_time, o.expected_end_time, o.start_time, o.
            order_date)
            END
        ) IS NOT NULL
        AND TIMEDIFF(
            DAY,
            MAX(
                CASE
                    WHEN o.order_state <> 'cancelled'
                    THEN COALESCE(o.actual_end_time, o.expected_end_time, o.start_time, o.
            order_date)
                END
            ),
            NOW()
```



```

        ) > 30 THEN 'idle'
        WHEN SUM(CASE WHEN o.order_state <> 'cancelled' THEN 1 ELSE 0 END) <= 1 THEN '
low_utilization'
        ELSE 'active'
    END AS utilization_flag
FROM items i
JOIN categories c ON c.category_id = i.category_id
LEFT JOIN orders o ON o.item_id = i.item_id
GROUP BY i.item_id, i.item_name, c.category_name, i.item_status
ORDER BY booked_days DESC, total_orders DESC, i.item_id;

```

Listing 16: Query 3: Item utilization and idle inventory identification

```

SELECT
    u.user_id,
    u.display_name,
    ROUND(u.reputation_score, 2) AS reputation_score,
    COUNT(o.order_id) AS total_orders,
    ROUND(COALESCE(SUM(CASE WHEN o.order_state <> 'cancelled' THEN o.total_price END), 0), 2)
    AS booked_value,
    ROUND(COALESCE(AVG(CASE WHEN o.order_state <> 'cancelled' THEN o.lease_time END), 0), 2)
    AS avg_lease_time,
    SUM(CASE WHEN o.order_state = 'late_return' THEN 1 ELSE 0 END) AS late_returns,
    SUM(CASE WHEN o.order_state = 'cancelled' THEN 1 ELSE 0 END) AS cancellations
FROM vw_users_masked u
JOIN orders o ON o.borrower_user_id = u.user_id
GROUP BY u.user_id, u.display_name, u.reputation_score
ORDER BY booked_value DESC, total_orders DESC, u.user_id;

```

Listing 17: Query 4: High-value borrower identification

```

SELECT
    u.user_id,
    u.display_name,
    ROUND(u.reputation_score, 2) AS reputation_score,
    COUNT(o.order_id) AS total_orders,
    SUM(CASE WHEN o.order_state = 'cancelled' THEN 1 ELSE 0 END) AS cancellation_count,
    SUM(CASE WHEN o.order_state = 'late_return' THEN 1 ELSE 0 END) AS late_return_count,
    ROUND(1.0 * SUM(CASE WHEN o.order_state = 'cancelled' THEN 1 ELSE 0 END) / COUNT(o.
    order_id), 3) AS cancellation_rate,
    ROUND(1.0 * SUM(CASE WHEN o.order_state = 'late_return' THEN 1 ELSE 0 END) / COUNT(o.
    order_id), 3) AS late_return_rate,
    ROUND(
        (1.0 * SUM(CASE WHEN o.order_state = 'cancelled' THEN 1 ELSE 0 END) / COUNT(o.
        order_id)) * 20 +
        (1.0 * SUM(CASE WHEN o.order_state = 'late_return' THEN 1 ELSE 0 END) / COUNT(o.
        order_id)) * 30 +
        CASE WHEN u.reputation_score < 4.2 THEN 10 ELSE 0 END
    , 2) AS risk_score
FROM vw_users_masked u
JOIN orders o ON o.borrower_user_id = u.user_id
GROUP BY u.user_id, u.display_name, u.reputation_score
HAVING COUNT(o.order_id) >= 3
ORDER BY risk_score DESC, late_return_rate DESC, cancellation_rate DESC, u.user_id;

```

Listing 18: Query 5: High-risk borrower identification

```

WITH labeled_orders AS (
    SELECT
        o.order_id,
        CASE
            WHEN u.reputation_score >= 4.7 THEN 'high_rep'
            WHEN u.reputation_score >= 4.3 THEN 'mid_rep'
            ELSE 'low_rep'
        END AS reputation_band,
        CASE
            WHEN o.lease_time <= 3 THEN 'short_lease'

```

```

        WHEN o.lease_time <= 7 THEN 'medium_lease'
        ELSE 'long_lease'
    END AS lease_band,
    o.order_state
FROM orders o
JOIN vw_users_masked u ON u.user_id = o.borrower_user_id
)
SELECT
    reputation_band,
    lease_band,
    COUNT(*) AS total_orders,
    SUM(CASE WHEN order_state = 'cancelled' THEN 1 ELSE 0 END) AS cancellations,
    SUM(CASE WHEN order_state = 'late_return' THEN 1 ELSE 0 END) AS late_returns,
    ROUND(1.0 * SUM(CASE WHEN order_state = 'cancelled' THEN 1 ELSE 0 END) / COUNT(*), 3) AS
    cancellation_rate,
    ROUND(1.0 * SUM(CASE WHEN order_state = 'late_return' THEN 1 ELSE 0 END) / COUNT(*), 3)
    AS late_return_rate
FROM labeled_orders
GROUP BY reputation_band, lease_band
ORDER BY cancellation_rate DESC, late_return_rate DESC, total_orders DESC;

```

Listing 19: Query 6: Late-return and cancellation pattern mining

E Security and Privacy SQL Implementation

This appendix presents representative SQL snippets for the security and privacy mechanisms described in Section 2.6. The snippets include password-hash constraints, masked views, role definitions, least-privilege grants, and database account controls.

E.1 Password Hash Constraint

The database adds a check constraint to ensure that stored user passwords follow the expected hash-marker format rather than being stored as plain text.

```

ALTER TABLE users
ADD CONSTRAINT chk_users_password_hash_format
CHECK (password_hash LIKE 'sha256$%');

```

E.2 Masked User and Payment Views

The following views expose only controlled representations of user and payment data. They support routine operation and analysis without requiring direct access to raw sensitive attributes.

```

CREATE OR REPLACE VIEW vw_users_masked AS
SELECT
    u.user_id,
    u.display_name,
    CONCAT(
        LEFT(SUBSTRING_INDEX(u.email, '@', 1), 2),
        '***@',
        SUBSTRING_INDEX(u.email, '@', -1)
    ) AS masked_email,
    CASE
        WHEN CHAR_LENGTH(u.phone) >= 4
        THEN CONCAT(LEFT(u.phone, 7), '***', RIGHT(u.phone, 2))
        ELSE '***'
    END AS masked_phone,
    u.registration_date,
    u.verification_status,
    u.reputation_score,
    u.account_status,
    u.deposit
FROM users u;

```

```

CREATE OR REPLACE VIEW vw_user_identity_masked AS
SELECT
    s.user_id,
    s.government_id_type,
    CONCAT(
        LEFT(s.real_legal_name, 1),
        REPEAT('*', GREATEST(CHAR_LENGTH(s.real_legal_name) - 1, 1))
    ) AS masked_legal_name,
    CONCAT(
        REPEAT('*', GREATEST(CHAR_LENGTH(s.government_id_number) - 4, 0)),
        RIGHT(s.government_id_number, 4)
    ) AS masked_government_id,
    s.verification_date
FROM sensitive_data s;

```

```

CREATE OR REPLACE VIEW vw_payment_methods_masked AS
SELECT
    pm.payment_method_id,
    pm.user_id,
    pm.method_type,
    pm.provider_name,
    pm.masked_account,
    pm.is_default,
    pm.status,
    pm.created_at
FROM payment_methods pm;

```

E.3 Role Definitions and Least-Privilege Grants

CampusRent defines four database roles. Each role receives only the permissions needed for its responsibility in the platform workflow.

```

CREATE ROLE IF NOT EXISTS 'admin';
CREATE ROLE IF NOT EXISTS 'app_user';
CREATE ROLE IF NOT EXISTS 'operator';
CREATE ROLE IF NOT EXISTS 'data_analyst';

GRANT ALL PRIVILEGES ON rental_platform.* TO 'admin';

```

```

GRANT SELECT ON rental_platform.categories TO 'app_user';
GRANT SELECT ON rental_platform.locations TO 'app_user';
GRANT SELECT ON rental_platform.items TO 'app_user';
GRANT SELECT, INSERT, UPDATE ON rental_platform.orders TO 'app_user';
GRANT SELECT, INSERT, UPDATE ON rental_platform.payment_methods TO 'app_user';
GRANT SELECT, INSERT ON rental_platform.users TO 'app_user';
GRANT INSERT ON rental_platform.sensitive_data TO 'app_user';
GRANT SELECT ON rental_platform.vw_user_auth TO 'app_user';
GRANT SELECT ON rental_platform.vw_users_masked TO 'app_user';
GRANT SELECT ON rental_platform.vw_payment_methods_masked TO 'app_user';

```

```

GRANT SELECT ON rental_platform.categories TO 'operator';
GRANT SELECT ON rental_platform.locations TO 'operator';
GRANT SELECT, INSERT, UPDATE ON rental_platform.items TO 'operator';
GRANT SELECT, UPDATE ON rental_platform.orders TO 'operator';
GRANT SELECT ON rental_platform.vw_users_masked TO 'operator';
GRANT SELECT ON rental_platform.vw_user_directory TO 'operator';
GRANT SELECT ON rental_platform.vw_payment_methods_masked TO 'operator';

```

```

GRANT SELECT ON rental_platform.categories TO 'data_analyst';
GRANT SELECT ON rental_platform.locations TO 'data_analyst';
GRANT SELECT ON rental_platform.items TO 'data_analyst';

```

```
GRANT SELECT ON rental_platform.orders TO 'data_analyst';
GRANT SELECT ON rental_platform.vw_users_masked TO 'data_analyst';
GRANT SELECT ON rental_platform.vw_payment_methods_masked TO 'data_analyst';
```

E.4 Database Accounts and Account Controls

The implementation also creates separate local database accounts and binds each account to its corresponding role. Password expiration, failed-login limits, and temporary lockout settings are applied at the database-account level.

```
CREATE USER IF NOT EXISTS 'admin_local'@'localhost'
  IDENTIFIED BY 'Admin_Local#2026_Strong';
CREATE USER IF NOT EXISTS 'operator_local'@'localhost'
  IDENTIFIED BY 'Operator_Local#2026_Strong';
CREATE USER IF NOT EXISTS 'app_local'@'localhost'
  IDENTIFIED BY 'App_Local#2026_Strong';
CREATE USER IF NOT EXISTS 'analyst_local'@'localhost'
  IDENTIFIED BY 'Analyst_Local#2026_Strong';
```

```
ALTER USER 'app_local'@'localhost'
  PASSWORD EXPIRE INTERVAL 180 DAY
  FAILED_LOGIN_ATTEMPTS 5
  PASSWORD_LOCK_TIME 2;

ALTER USER 'admin_local'@'localhost'
  PASSWORD EXPIRE INTERVAL 180 DAY
  FAILED_LOGIN_ATTEMPTS 5
  PASSWORD_LOCK_TIME 2;

ALTER USER 'operator_local'@'localhost'
  PASSWORD EXPIRE INTERVAL 180 DAY
  FAILED_LOGIN_ATTEMPTS 5
  PASSWORD_LOCK_TIME 2;

ALTER USER 'analyst_local'@'localhost'
  PASSWORD EXPIRE INTERVAL 180 DAY
  FAILED_LOGIN_ATTEMPTS 5
  PASSWORD_LOCK_TIME 2;
```

```
GRANT 'app_user' TO 'app_local'@'localhost';
GRANT 'admin' TO 'admin_local'@'localhost';
GRANT 'operator' TO 'operator_local'@'localhost';
GRANT 'data_analyst' TO 'analyst_local'@'localhost';

SET DEFAULT ROLE 'app_user' TO 'app_local'@'localhost';
SET DEFAULT ROLE 'admin' TO 'admin_local'@'localhost';
SET DEFAULT ROLE 'operator' TO 'operator_local'@'localhost';
SET DEFAULT ROLE 'data_analyst' TO 'analyst_local'@'localhost';
```

The complete implementation is provided in `database/security_setup.sql`.

F User Interface Implementation

This appendix summarizes how the CampusRent user interface is implemented at the engineering level. The appendix does not include source code; the complete UI implementation is provided in the `ui/` directory of the submitted project files.

F.1 Technology Stack

The CampusRent interface is implemented as a local web application. The back-end is built with Flask, while the database connection is handled through MySQL. The front-end pages are rendered through

server-side HTML templates, with static assets used for styling and page layout. This architecture was selected because it is lightweight, easy to run locally, and sufficient for demonstrating how the database supports realistic platform workflows.

The UI is not based on mock data. Instead, it connects directly to the `rental_platform` MySQL database. Therefore, user actions in the interface correspond to actual database operations, such as reading inventory records, creating orders, updating return status, and displaying analytical query results.

F.2 Project Organization

The UI-related files are organized under the `ui/` directory. The main application code is stored in `ui/app/`. This folder contains the Flask application, HTML templates, static style files, and scripts for running the local interface. The one-click launcher is stored separately under `ui/launcher/`. It is used to start the local demo environment more conveniently.

In the final project structure, the UI works together with the database and SQL folders. The database is initialized through `database/init.sql`, security and role settings are applied through `database/security_setup.sql`, and the UI then connects to the initialized MySQL database to provide interactive access to the platform.

F.3 Application Architecture

The UI follows a role-aware structure. Regular users access the platform through the ordinary login and registration pages. Staff users enter through a separate staff portal and are redirected to different consoles according to their role. The four major interface areas are:

- the regular user interface, which supports account login, inventory browsing, item details, order creation, and rental history;
- the operator console, which supports item intake, return confirmation, and inventory maintenance status updates;
- the admin console, which supports schema inspection, ER diagram display, relational table browsing, and paginated data previews;
- the analyst console, which supports read-only dataset browsing, CSV export for approved datasets, and display of analytical SQL results.

This organization mirrors the database role model. The UI therefore does not only provide navigation; it also demonstrates how different database users should interact with different parts of the system.

F.4 Connection to the Database

The Flask application connects to MySQL using role-specific database accounts. For example, the admin pages use the administrative database account, the operator pages use the operator account, and the analyst pages use the analyst account. This means that the UI is aligned with the database permission model. Even if a page attempted to access data outside its role, the underlying database grants would restrict the operation.

The regular user pages query inventory, category, location, and order data to support the rental workflow. Operator actions update item and order records. Admin pages query metadata and table records to display schema-level information. Analyst pages query approved base tables and masked views to present analytical results. In this way, each UI module corresponds to a clearly defined set of database operations.

F.5 Local Demo Launcher

To make the prototype easier to run, the project includes a local launcher under `ui/launcher/`. The launcher is designed to start the local MySQL service, prepare the database environment, start the Flask application, and open the UI in a browser. This allows the database demo to be used as a local application rather than requiring the user to manually start every component.

The launcher is primarily intended for demonstration. For development or manual testing, the same application can also be started through the scripts in `ui/app/`, after the MySQL database has been initialized.

F.6 Implementation Files

The most relevant UI implementation files are:

- `ui/app/app.py`: main Flask application, route definitions, database access logic, role checks, and workflow handlers;
- `ui/app/templates/`: HTML templates for user pages, staff pages, admin pages, and analyst pages;
- `ui/app/static/`: CSS and static assets used by the interface;
- `ui/app/run_ui.ps1`: script for starting the UI manually in the local environment;
- `ui/launcher/`: one-click launcher source and executable files for the local demo.

The complete implementation is provided in the `ui/` directory.